

NLP amaliyotida MLOps

Tabiiy tilni qayta ishlash — 16-ma'ruza (yakuniy)

A.A. Abdulali

11:30–12:50 · mavzu №16

10-iyul 2026

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarish muhitiga joriy etish bosqichlari
- 3 API yaratish va Docker bilan konteynerlashtirish
- 4 Model monitoringi va versiyalash
- 5 CI/CD pipeline
- 6 Xulosa, capstone himoyasi va keyingi yo'l

Takrorlash: o'tgan darsdan ikki savol

1-savol

L15/P15 da hujjat yordamchisi agentini **qurdik**. U hozir lokal kompyuterda ishlaydi. Uni foydalanuvchilarga qanday yetkazamiz?

2-savol

Model bir oydan keyin ham bir xil aniqlikda ishlaydimi? Yangi ma'lumot kelsa nima qilamiz?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L15/P15 da hujjat yordamchisi agentini **qurdik**. U hozir lokal kompyuterda ishlaydi. Uni foydalanuvchilarga qanday yetkazamiz?

2-savol

Model bir oydan keyin ham bir xil aniqlikda ishlaydimi? Yangi ma'lumot kelsa nima qilamiz?

1-javob

Modelni **API ko'rinishidagi xizmat** sifatida joylashtiramiz va **Docker konteyneriga** o'raymiz. Bu P16 amaliyotining asosiy g'oyasi.

2-javob

Yo'q. Model sifati vaqt o'tishi bilan pasayishi mumkin: bunga **siljish** (*drift*) deyiladi. Uni monitoring, versiyalash va qayta o'qitish bilan boshqaramiz.

Kalit fikr: MLOps (*Machine Learning Operations*) — modelni ishonchli, kuzatiladigan va yangilanadigan xizmatga aylantirish amaliyoti.

Dars yakunida nimalarni bajara olasiz?

- 1 Modelni ishlab chiqarish muhitiga joriy etish bosqichlarini (*train* → *package* → *serve* → *monitor*) **tushuntira olasiz**.
- 2 FastAPI va Docker yordamida modelni API (*Application Programming Interface*) ko'rinishidagi xizmatga aylantirish g'oyasini **tushuntira olasiz**.
- 3 Model monitoringi, siljish (*drift*) va versiyalash nima uchun kerakligini **tushuntira olasiz**.
- 4 CI/CD (*Continuous Integration/Continuous Delivery*) konveyeri NLP loyihasini avtomatlashtirishda qanday ishlashini **tushuntira olasiz**.

Oldindan talab: P15 dagi agent g'oyasi, M4/P16 dagi FastAPI/Docker amaliyoti, JSON formati va model fayllari bilan ishlash. Bu dars amaliy ishda ko'rilgan jarayonni MLOps nuqtayi nazaridan umumlashtiradi.

Bugungi dars rejasi

Bo'lim	Mavzu	Asosiy g'oya
1	Ishlab chiqarish muhitiga joriy etish bosqichlari	<i>train</i> → <i>package</i> → <i>serve</i> → <i>monitor</i>
2	API yaratish va Docker bilan konteynerlashtirish	FastAPI + Docker
3	Monitoring va versiyalash	siljish (<i>drift</i>), model reyestri
4	CI/CD konveyeri	test → build → deploy
–	Xulosa, yakuniy loyiha (<i>capstone</i>) himoyasi va keyingi yo'l	Kurs yakuni

Bu dars — kurs yakuni. Kurs davomida qurilgan “O'zbek hujjat yordamchisi” prototipini ishonchli xizmatga yaqinlashtirish: API, Docker, monitoring va CI/CD orqali joriy etish mantiqini ko'ramiz.

Bugungi muammo: “menda ishlaydi” yetarli emas

Notebook prototipi qayerda qiynaladi?

Faqat bitta muhitda ishlaydi

Model Kaggle yoki lokal kompyuterda ishlashi mumkin, lekin boshqa serverda kutubxona va versiya farqi sababli natija takrorlanmasligi mumkin.

Vaqt o'tishi bilan sifati pasayishi mumkin

Ma'lumot taqsimoti o'zgarsa (*data drift*), model aniqligi pasayishi mumkin. Monitoring bo'lmasa, bu holat kech aniqlanadi.

Bugungi yechim: MLOps

- 1 Modelni **artefakt** sifatida saqlash va **API** orqali xizmatga chiqarish.
- 2 **Docker** yordamida kutubxonalar, versiyalar va ishga tushirish muhitini barqarorlashtirish.
- 3 **Monitoring**, versiyalash va **CI/CD** orqali xizmatni kuzatish hamda yangilashni avtomatlashtirish.

MLOps (*Machine Learning Operations*) — ML modelni ishonchli, takror ishlab bo'ladigan va yangilanadigan xizmatga aylantirish amaliyoti.

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarish muhitiga joriy etish bosqichlari
- 3 API yaratish va Docker bilan konteynerlashtirish
- 4 Model monitoringi va versiyalash
- 5 CI/CD pipeline
- 6 Xulosa, capstone himoyasi va keyingi yo'l

Intuitsiya: retsept emas, restoran

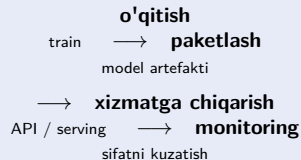
Notebook prototipi

Jupyter notebook — **retsept**: odatda bir muhitda, tajriba va tekshirish uchun ishlaydi.

Ishlab chiqarish xizmati

Production muhit — **restoran**: ko'p foydalanuvchi, barqaror sifat, tezkor javob va doimiy kuzatuv talab qiladi.

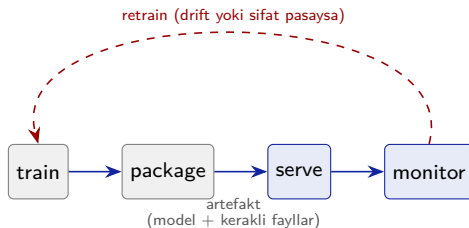
Shuning uchun o'qitish va xizmatga chiqarish alohida bosqichlarga ajratiladi.



Oldingi amaliyotlarda model yoki indeks saqlangan edi. Bu darsda shu artefakti yuklab, API orqali xizmat sifatida ishlatish mantiqini ko'ramiz.

Deploy bosqichlari: rasmiy ta'rif

Asosiy zanjir: $train \rightarrow package \rightarrow serve \rightarrow monitor \rightarrow retrain$



kulrang = davriy / zaruratga ko'ra

ko'k = xizmat ishlayotgan bosqichlar

Mazmuni: *train* — modelni o'qitish; *package* — model artefakti, bog'liqliklar va kerakli fayllarni tayyorlash; *serve* — modelni API orqali ishlatish; *monitor* — sifat, kechikish va xatolarni kuzatish; *retrain* — kerak bo'lsa modelni qayta o'qitish.

Asoslash: nega bosqichlarni ajratamiz?

- 1 **O'qitish qimmat.** Modelni har bir so'rovda qayta o'qitib bo'lmaydi.
- 2 **Model artefaktga aylantiriladi.** O'qitilgan model, tokenizer/preprocessing va sozlamalar saqlanadi.
- 3 **Xizmat bosqichi tez ishlashi kerak.** Server ishga tushganda artefaktni yuklaydi; har so'rovda esa faqat bashorat (*inference*) bajariladi.
- 4 **Monitoring siklni yopadi.** Sifat, kechikish va xatolar kuzatiladi; muammo aniqlansa, model qayta o'qitiladi yoki yangi versiya chiqariladi.

Ajratilgan oqim

train → *artifact* → *serve*
→ *monitor* → *retrain*

Xulosa: bosqichlarni ajratish xizmatni tezroq, takrorlanuvchanroq va boshqariladigan qiladi.

Qo'lda misol: modelni bir marta yuklash

Sharoit: model fayli 400 MB; yuklash vaqti ≈ 2 s, bitta bashorat vaqti ≈ 20 ms.

1) Har so'rovda yuklash:

$2000 + 20 = 2020$ ms/so'rov = 2.02 s/so'rov. Bu juda sekin.

2) Server startida bir marta yuklash:

barqaror ish rejimida har so'rov ≈ 20 ms.

Taqqoslash:

$\frac{2020}{20} = 101 \Rightarrow$ amalda **taxminan** $100\times$ **tezroq**.

Model server ishga tushganda **xotiraga bir marta yuklanadi**; keyingi so'rovlar shu nusxadan foydalanadi. **Eslatma:** yuklash server startini sekinlashtiradi; agar lazy loading ishlatilsa, birinchi so'rov ham sekin bo'lishi mumkin.

Bitta so'rov uchun kechikish



Sizning vazifangiz: soniyadagi yukni hisoblash

Savol

Server ishga tushganda model xotiraga bir marta yuklangan. Har bir bashorat ≈ 20 ms davom etadi. Bitta jarayon ketma-ket ishlasa, server bir soniyada taxminan nechta so'rovga javob bera oladi?

Sizning vazifangiz: soniyadagi yukni hisoblash

Savol

Server ishga tushganda model xotiraga bir marta yuklangan. Har bir bashorat ≈ 20 ms davom etadi. Bitta jarayon ketma-ket ishlasa, server bir soniyada taxminan nechta so'rovga javob bera oladi?

Javob

$$\text{so'rov/soniya} \approx \frac{1000 \text{ ms}}{20 \text{ ms}} = \mathbf{50}$$

Demak, bitta jarayon ketma-ket ishlaganda nazariy maksimum ≈ 50 so'rov/soniya. Amalda API, tarmoq va navbat (*queue*) xarajatlari sababli qiymat biroz past bo'lishi mumkin.

Ko'proq yuk kerak bo'lsa, bir nechta **worker/replica** ishga tushiriladi yoki mos holatda so'rovlar **batch** qilib qayta ishlanadi. Bu — **masshtablash**.

Kod ↔ jarayon: modelni bir marta yuklash

```
# server startida BIR MARTA
clf = FineTunedClassifier()
clf.load("m13_model")      # artefaktni yuklash

# har so'rovda: faqat inference
def predict(text):
    return clf.predict(text)
```

Kod → MLOps ma'nosi

Kod	Vazifa
load	serve startida artefaktni xoti- raga yuklash
predict	har so'rovda bashorat qilish

G'oya: load bir marta bajariladi; predict esa har so'rovda ishlaydi. Shuning uchun so'rov vaqti asosan *inference* vaqtiga teng.

Deploy tuzog'i: train-serve tafovuti

Muammo: training-serving skew

Train paytida ishlatilgan **preprocessing** xizmatda aynan takrorlanmasa, model boshqa ko'rinishdagi kirishni ko'radi.

Misol: tokenizer, normalizatsiya yoki TF-IDF/vectorizer versiyasi mos kelmasa, bashorat ishonchsiz bo'ladi.

Intuitsiya: model xom matnni emas, train paytida hosil qilingan aynan shu **xususiyatlar ko'rinishini** o'rgangan.

Yechim

- 1 Bir xil preprocessing modulidan foydalanish.
- 2 Tokenizer/vectorizer va konfiguratsiyani model bilan birga saqlash.
- 3 Model, kod va preprocessing versiyasini birga belgilash.
- 4 Train va serve chiqishini solishtiruvchi oddiy test yozish.

Amaliy qoida: model artefakti yolg'iz saqlanmaydi; u tokenizer, preprocessing kodi va versiya ma'lumotlari bilan birga yuradi.

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarish muhitiga joriy etish bosqichlari
- 3 API yaratish va Docker bilan konteynerlashtirish**
- 4 Model monitoringi va versiyalash
- 5 CI/CD pipeline
- 6 Xulosa, capstone himoyasi va keyingi yo'l

Intuitsiya: modelga kirish nuqtasi va barqaror muhit

API (*Application Programming Interface*) — modelga tashqi tizimlar murojaat qiladigan standart **kirish nuqtasi**.

Masalan, boshqa dastur HTTP/JSON so'rov yuboradi va modeldan bashorat javobini oladi.

Docker — modelni ishga tushirish muhitini barqarorlashtirish usuli.

Kod, kutubxonalar va model fayllari bir xil muhitda ishlashi uchun konteynerdan foydalaniladi.

Kalit fikr: FastAPI modelga kirish nuqtasini beradi; Docker esa xizmatni boshqa serverda qayta ishga tushirishni osonlashtiradi.

Minimal xizmat oqimi

Foydalanuvchi / dastur

HTTP/JSON so'rov



FastAPI endpoint

POST /predict



NLP model

bashorat / inference



JSON javob

Bu slayddagi savol: foydalanuvchi modelga qanday murojaat qiladi va xizmat boshqa muhitda qanday barqaror ishlaydi?

API va konteyner: rasmiy ta'rif

API: *request* → *response*

Docker image: *base image* + *deps* + *model* + *kod*

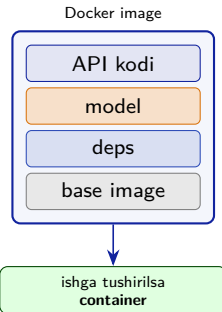
Container: *ishga tushgan image*

API — HTTP endpoint orqali modelga murojaat qilish usuli. Masalan: POST /predict.

Image — Docker tasviri: bazaviy muhit(base image), kutubxonalar(deps), model fayllari va ilova kodining paketlangan ko'rinishi.

Container — image ning **ishga tushgan nusxasi**. Ya'ni image tayyor shablon bo'lsa, container shu shablon asosida ishlayotgan jarayondir.

Kalit farq: image quriladi va saqlanadi; container esa shu image asosida ishga tushiriladi.



Eslatma: Docker takrorlanuvchan ishga tushirishni osonlashtiradi, lekin apparat, GPU drayveri va tashqi servislar ham muhim.

Kod ↔ xizmat: minimal FastAPI endpoint

```
from fastapi import FastAPI
from pydantic import BaseModel

class PredictRequest(BaseModel):
    text: str

app = FastAPI()
model = load_model("model/")  # server startida bir marta

@app.post("/predict")
def predict(req: PredictRequest):
    pred = model.predict(req.text)
    return {
        "label": pred.label,
        "confidence": pred.score
    }
```

Kod → MLOps ma'nosi

Kod	Vazifa
BaseModel	kirish sxemasi
/predict	HTTP endpoint
load_model	artefaktni yuklash
return	JSON javob

API matnni JSON so'rov orqali qabul qiladi va model bashoratini JSON javob sifatida qaytaradi. Shu sababli model boshqa dasturlar bilan ulanishi mumkin.

Asoslash: nega Docker kerak?

Muammo

Model kutubxona va versiyalarga bog'liq: torch, scikit-learn, tokenizer, config va boshqa fayllar.

Agar train muhiti va server muhiti farq qilsa, model ishlamasligi yoki natija takrorlanmasligi mumkin.

Docker image odatda quyidagilarni bir joyga yig'adi:

base image + deps + kod + model

Docker nima beradi?

- 1 Kutubxonalar va ishga tushirish muhitini aniqroq belgilaydi.
- 2 Ilova kodi, model fayllari va sozlamalarni bitta birlikka yig'adi.
- 3 Shu image turli serverlarda o'xshash muhitda ishga tushadi.

Xulosa: API modelga kirish nuqtasini beradi; Docker esa xizmatni barqaror va takrorlanuvchan muhitda ishga tushirishga yordam beradi.

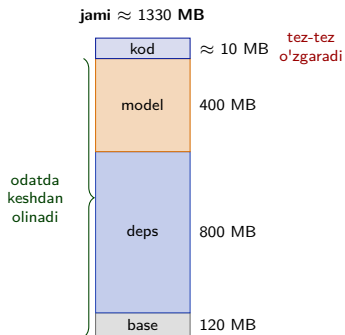
Qo'lda misol: Docker image qatlamlari

Soddalashtirilgan misol: *base image* (≈ 120 MB) + *kutubxonalar* (≈ 800 MB) + *model* (≈ 400 MB) + *kod* (≈ 10 MB).

Jami image hajmi ≈ 1330 MB. Docker image bir nechta **qatlamlardan** tuziladi.

Agar `requirements.txt` o'zgarmasa, kutubxonalar qatlami odatda **keshdan olinadi**. Agar faqat kod o'zgarsa, pastdagi qatlamlar saqlanib qoladi, **o'zgargan qatlam va undan keyingilar** qayta quriladi.

Amaliy qoida: avval `requirements.txt` ni nusxalab kutubxonalarni o'rnating, **kodni esa oxirida** nusxalang. Shunda kod tez-tez o'zgarganda kesh samaraliroq ishlaydi.



Sizning vazifangiz: faqat kod o'zgardi

Savol

Siz faqat ilova kodini o'zgartirdingiz. Kutubxonalar va model fayllari o'zgarmadi. Qaysi qatlamlar keshdan olinadi, qaysi qatlam qayta quriladi?

Sizning vazifangiz: faqat kod o'zgardi

Savol

Siz faqat ilova kodini o'zgartirdingiz. Kutubxonalar va model fayllari o'zgarmadi. Qaysi qatlamlar keshdan olinadi, qaysi qatlam qayta quriladi?

Javob

Agar Dockerfile tartibi

base → deps → model → kod

bo'lsa, **base**, **deps** va **model** qatlamlari keshdan olinadi; faqat **kod** qatlami qayta quriladi.

Qoida: Docker'da o'zgargan qatlam va undan keyingi qatlamlar qayta quriladi. Shuning uchun kam o'zgaradigan narsalar yuqoriroqqa, tez o'zgaradigan kod esa pastroqqa qo'yiladi.

Kod ↔ bosqich: Dockerfile

```
FROM python:3.11-slim          # base image

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt # deps

COPY model/ ./model/          # model artefakti
COPY app.py ./                 # tez o'zgaradigan kod

EXPOSE 8000
CMD ["uvicorn", "app:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

Qator → qatlam

Qator	Vazifa
FROM	bazaviy muhit
RUN pip	kutubxonalar
COPY model	model artefakti
COPY app	ilova kodi
CMD	ishga tushirish

Kesh g'oyasi: kam o'zgaradigan qatlamlar yuqorida, tez o'zgaradigan app.py pastda turadi. Shunda faqat kod o'zgarsa, base, deps va model qatlamlari keshdan olinadi.

Docker tuzog'i: katta image va maxfiy kalitlar

Muammo

Keraksiz fayllar — datasetlar, cache, notebooklar va loglar — image hajmini kattalashtiradi.

API kaliti, token yoki parolni image ichiga yozish esa **xavfsizlik xatosi**: image tarqalsa, maxfiy ma'lumot ham tarqaladi.

Yechim

- 1 `.dockerignore` bilan keraksiz fayllarni image ga kiritmang.
- 2 `python:3.11-slim` kabi yengil base image ishlating.
- 3 Maxfiy kalitlarni kod yoki image ichiga yozmang.
- 4 Lokal ishda environment o'zgaruvchilari, production da esa secrets manager ishlating.

Eslatma: Docker muhitni takrorlashni osonlashtiradi, lekin **mutlaq bir xil natijani kafolatlamaydi**. CPU arxitekturasini (x86/ARM), GPU drayveri, CUDA versiyasi yoki tashqi servislar farq qilsa, ishlash tezligi va ba'zan natija ham farq qilishi mumkin.

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarish muhitiga joriy etish bosqichlari
- 3 API yaratish va Docker bilan konteynerlashtirish
- 4 Model monitoringi va versiyalash**
- 5 CI/CD pipeline
- 6 Xulosa, capstone himoyasi va keyingi yo'l

Intuitsiya: model eskiradi, uni kuzatib borish kerak

Nega monitoring kerak?

Foydalanuvchi matnlari vaqt o'tishi bilan o'zgaradi: yangi so'zlar, yangi mavzular, yangi yozish uslublari paydo bo'ladi.

Model esa avvalgi ma'lumot taqsimotida o'qitilgan. Shuning uchun ma'lumot siljishi (*drift*) yuz bersa, model sifati pasayishi mumkin.

Yechim: metrikalarni kuzatish, muammo bo'lsa ogohlantirish, kerak bo'lsa yangi model versiyasiga o'tish.

kuzatish
latency, xatolar, sifat metrikalari



ogohlantirish
threshold buzilsa



versiyalash / rollback
yangi model yoki eski barqaror versiya

Model reyestri (*model registry*) v1, v2, ... versiyalarni, metrikalarni va kerakli fayllarni saqlaydi. Xato bo'lsa, eski barqaror versiyaga qaytish (*rollback*) mumkin.

Monitoring va drift: rasmiy ta'rif

Oddiy ogohlantirish qoidasi: $\Delta = M_{\text{base}} - M_{\text{joriy}}$, $\Delta \geq \tau \Rightarrow$ ogohlantirish

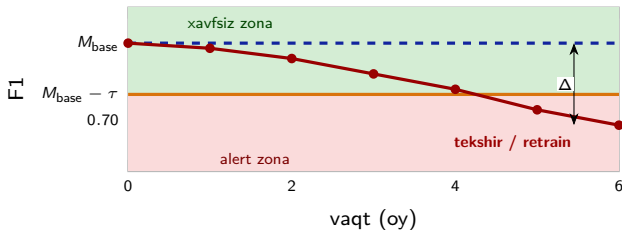
Bu yerda: M_{base} — boshlang'ich metrika, M_{joriy} — joriy o'lchov, Δ — pasayish, τ — chegaraviy qiymat.

Bu qoida **F1**, **accuracy** kabi “katta bo'lsa yaxshi” metrikalar uchun qulay.

Agar $\Delta \geq \tau$ bo'lsa, avval **ogohlantirish** beriladi; so'ng sabab tekshiriladi va kerak bo'lsa **retrain** yoki **rollback** qilinadi.

Eslatma: yorliqlar bo'lmasa, bevosita sifatni emas, kiruvchi ma'lumotdagi *data drift* indikatorlarini (masalan, token taqsimoti, uzunlik, yangi so'zlar ulushi) kuzatamiz.

Metrika pasaysa ogohlantirish beriladi



Monitoring ikki qatlam: server holati va model sifati

1) Texnik monitoring

Metrika	Nimani ko'rsatadi
latency	javob kechikishi (ms)
throughput	so'rov/soniya
error rate	xato javoblar ulushi
uptime	xizmat mavjudligi

2) Model sifati monitoringi

Metrika	Nimani ko'rsatadi
kirish taqsimoti	data drift: $P(X)$ o'zgardimi
bashorat taqsimoti	chiqishlar siljidimi
confidence	model ishonchi barqarormi
F1 / accuracy	sifat pasaydimi (yorliq bo'lsa)

Asosiy farq — nega ikkalasi kerak?

Server “yashil” bo'lishi model to'g'ri ishlayotganini kafolatlamaydi: API tez javob berishi mumkin, lekin model noto'g'ri bashorat qilayotgan bo'lishi mumkin.

Data drift: kirish taqsimoti $P(X)$ o'zgaradi, ko'pincha yorliqsiz kuzatiladi. **Concept drift:** bog'liqlik $P(Y|X)$ o'zgaradi; odatda yorliqli ma'lumot va metrika pasayishi orqali tasdiqlanadi.

Asoslash: sifat pasayishini qanday aniqlaymiz?

- 1 **Baseline metrikani saqlaymiz.** O'qitishdan keyin nazorat to'plamidagi sifat qayd etiladi: $F_{1,base}$.
- 2 **Joriy metrikani o'lchaymiz.** Production davrida yangi yorliqlangan namunada sifat hisoblanadi: $F_{1,joriy}$.

- 3 **Pasayishni hisoblaymiz.**

$$\Delta = F_{1,base} - F_{1,joriy}$$

- 4 **Chegara bilan solishtiramiz.** Agar $\Delta \geq \tau$ bo'lsa, ogohlantirish beriladi va sabab tekshiriladi.

Qaror oqimi:

$F_{1,base}$ va $F_{1,joriy}$ ni solishtiramiz



$$\Delta = F_{1,base} - F_{1,joriy}$$



$$\Delta \geq \tau \text{ bo'lsa: alert}$$



sababni tekshirish → retrain yoki
rollback

Xulosa: monitoring sifatning sezilmay pasayishini erta ko'rsatadi. Bu darhol retrain degani emas; avval sabab aniqlanadi.

Eslatma: bu qoida F1/accuracy kabi “katta bo'lsa yaxshi” metrikalar uchun qulay. Yorliqlar bo'lmasa, bevosita sifat emas, data drift indikatorlari alohida kuzatiladi.

Qo'lda misol: monitoring chegarasi

Berilgan: baseline $F_{1,base} = 0.85$; bir oydan keyin $F_{1,joriy} = 0.70$; chegara $\tau = 0.10$.

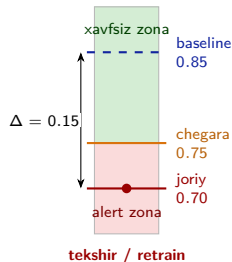
Hisob:

$$\begin{aligned}\Delta &= F_{1,base} - F_{1,joriy} \\ &= 0.85 - 0.70 = 0.15 \\ \Delta &= 0.15 > 0.10 = \tau\end{aligned}$$

Qaror: chegara buzildi $\Rightarrow$ ogohlantirish beriladi. Keyin sabab tekshiriladi: data drift, preprocessing xatosi, label sifati yoki serverdagi muammo bo'lishi mumkin.

Agar pasayish yangi ma'lumot taqsimoti bilan bog'liq bo'lsa, retrain qilinadi; yangi model yomon ishlasa, rollback qilinadi.

Chegara bilan solishtirish



Sizning vazifangiz: ogohlantirish kerakmi?

Savol

Baseline $F_{1,\text{base}} = 0.88$; joriy $F_{1,\text{joriy}} = 0.82$; chegara $\tau = 0.10$.

$\Delta = ?$ **Ogohlantirish ishga tushadimi?**

Sizning vazifangiz: ogohlantirish kerakmi?

Savol

Baseline $F_{1,base} = 0.88$; joriy $F_{1,joriy} = 0.82$; chegara $\tau = 0.10$.

$\Delta = ?$ **Ogohlantirish ishga tushadimi?**

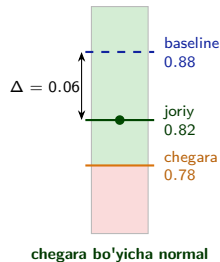
Javob

$$\Delta = F_{1,base} - F_{1,joriy} = 0.88 - 0.82 = 0.06$$

$$0.06 < 0.10 = \tau \Rightarrow \text{ogohlantirish yo'q}$$

Pasayish bor, lekin chegara buzilmadi. Demak, holatni kuzatishda davom etamiz. τ juda past qo'yilsa, behuda ogohlantirishlar ko'payadi; juda yuqori qo'yilsa, muammo kech aniqlanadi.

Chegara bilan solishtirish



Kod $\leftrightarrow$ formula: sifat pasayishini tekshirish

```
BASELINE_F1 = 0.85
TAU = 0.10

def check_quality_drop(current_f1):
    delta = BASELINE_F1 - current_f1

    if delta >= TAU:
        return "alert"      # sababni tekshir

    return "ok"

print(check_quality_drop(0.70)) # alert
```

Kod $\rightarrow$ formula

Kod	Formula
delta	Δ
BASELINE_F1	$F_{1,base}$
current_f1	$F_{1,joriy}$
delta >= TAU	$\Delta \geq \tau$

Davriy ish bu funktsiyani yangi yorliqlangan namunada ishga tushiradi. alert chiqsa, avval sabab tekshiriladi; kerak bo'lsa retrain yoki rollback qilinadi.

Versiyalash tuzog'i: eski model saqlanmagan

Muammo

Yangi model production muhitida yomon ishlasa, lekin eski barqaror versiya saqlanmagan bo'lsa, tezda ortga qaytish (*rollback*) qiyin bo'ladi.

Natijada xizmat sifati pasaygan holatda qolishi mumkin.

Tuzoq: faqat `model.pt` yoki `model.pkl` faylini almashtirish yetarli emas; u qaysi kod, qaysi tokenizer va qaysi data bilan olingani ham muhim.

Yechim: model reyestri

Har bir versiya uchun quyidagilar saqlanadi:

- model artefakti: `v1`, `v2`, ...;
- tokenizer/preprocessing va config;
- metrikalar, sana, kod/data versiyasi;
- holat: *staging*, *production*, *archived*.

Rollback g'oyasi: `v2` yomon ishlasa, xizmat tezda eski barqaror `v1` versiyasiga qaytariladi.

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarish muhitiga joriy etish bosqichlari
- 3 API yaratish va Docker bilan konteynerlashtirish
- 4 Model monitoringi va versiyalash
- 5 CI/CD pipeline**
- 6 Xulosa, capstone himoyasi va keyingi yo'l

Intuitsiya: qo'lda emas, avtomatik pipeline

Har bir o'zgarishdan keyin testlarni qo'lda ishga tushirish, Docker image qurish va xizmatni yangilash sekin hamda xatoga moyil.

CI/CD pipeline — kod o'zgarganda tekshirish, qurish va joylashtirish bosqichlarini avtomatik bajaradigan jarayon.

CI (*Continuous Integration*) — kodni tekshirish: testlar va build.

CD (*Continuous Delivery/Deployment*) — tayyor versiyani yetkazish yoki muhitga joylashtirish.

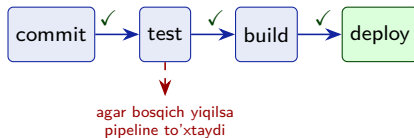
Pipeline oqimi



Biz amaliyotlarda yozgan assertlar — CI testlarining sodda ko'rinishi. Pipeline ularni avtomatik tekshiradi.

CI/CD: rasmiy ta'rif

CI/CD — koddagi o'zgarishlarni avtomatik tekshirish, qurish va muhitga joylashtirishni boshqaradigan jarayon.



Bosqichlar: *commit* — kodni repozitoriyga yuborish; *test* — avtomatik tekshiruvlar (assert, unit test); *build* — artefakt yoki Docker image qurish; *deploy* — tayyor versiyani staging yoki production muhitiga joylashtirish.

Asoslash: nega avtomatlashtirish kerak?

Qo'lda bajarilsa

Har o'zgarishdan keyin test, build va deploy bosqichlarini qo'lda bajarish:

- sekin va takroriy ish;
- inson xatosiga moyil;
- xatoli kodni kech aniqlaydi.

CI/CD pipeline bo'lsa

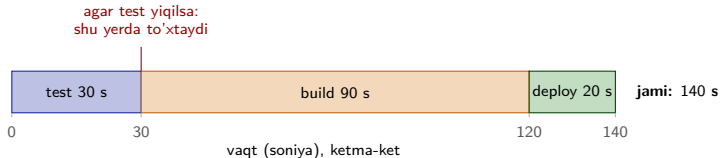
Har bir commitdan keyin jarayon avtomatik ishlaydi:

- 1 testlar ishga tushadi;
- 2 testlar o'tsa, Docker image quriladi;
- 3 build muvaffaqiyatli bo'lsa, deploy bajariladi;
- 4 bosqich yiqilsa, pipeline to'xtaydi.

Xulosa: CI/CD sifat nazorat bosqichlarini avtomatlashtiradi. Kursdagi assertlar esa shu avtomatik testlarning sodda ko'rinishi: kod o'zgarsa, ular xatoni erta ushlashga yordam beradi.

Qo'lda misol: pipeline bosqichlari

Faraz: pipeline ketma-ket ishlaydi. Test 30 s, build 90 s, deploy 20 s.



To'liq muvaffaqiyatli pipeline: $30 + 90 + 20 = 140$ s.

Agar test yiqilsa: pipeline taxminan 30 s da to'xtaydi, build va deploy bajarilmaydi. Shuning uchun **xatoli kod** ishlab chiqarish muhitiga chiqmaydi.

Asosiy foyda: dasturchi xato haqida tezkor qayta aloqa oladi. Muammo bir necha daqiqada bilinadi, kunlab kechikmaydi.

Eslatma: bu soddalashtirilgan ketma-ket misol. Amalda ayrim testlar parallel ishlashi mumkin.

Sizning vazifangiz: testdan o'tmadi

Savol

Pipeline jarayonida test bosqichi muvaffaqiyatsiz tugadi: bitta assert o'tmadi.

Keyingi bosqichlar — build va deploy — bajariladimi?

Sizning vazifangiz: testdan o'tmadi

Savol

Pipeline jarayonida test bosqichi muvaffaqiyatsiz tugadi: bitta assert o'tmadi.

Keyingi bosqichlar — build va deploy — bajariladimi?

Javob

Yo'q. Pipeline test bosqichida to'xtaydi; **build** va **deploy** bajarilmaydi.

Sababi: testdan o'tmagan koddan Docker image qurilsa va u production muhitiga chiqarilsa, xatoli versiya foydalanuvchiga yetib borishi mumkin.

Keyingi qadam: xatoni tuzatish → qayta commit qilish → pipeline testlarni yana ishga tushiradi.

Xulosa: assert va unit testlar CI/CD pipeline dagi birinchi sifat nazorati hisoblanadi.

Kod ↔ bosqich: CI/CD pipeline (YAML)

```
# .github/workflows/ci.yml
on:
  push:
    branches: [main]

jobs:
  pipeline:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pytest
      - run: docker build -t nlp-api:latest .
      - run: ./deploy.sh
```

Qator → bosqich

Qator	Bosqich
checkout	kodni olish
pytest	test
docker build	build
deploy.sh	deploy

main branchga push qilinganda pipeline avtomatik ishga tushadi. Biror bosqich muvaffaqiyatsiz bo'lsa, keyingi bosqichlar bajarilmaydi.

Eslatma: production deploy odatda maxfiy kalitlar va ruxsat nazorati bilan bajariladi.

CI/CD tuzog'i: testlarsiz pipeline

Muammo: faqat deploy avtomatik

Agar pipeline faqat deploy qilsa, lekin testlardan o'tkazmasa, xatoli kod ishlab chiqarish muhitiga tezroq chiqib ketadi.

Demak, avtomatlashtirishning o'zi yetarli emas: u **sifat nazorati** bilan birga bo'lishi kerak.

Noto'g'ri oqim:

`commit → build → deploy`

Yechim

Deploydan oldin majburiy test bosqichi qo'yiladi:

`commit → test → build → deploy`

Test muvaffaqiyatsiz bo'lsa, pipeline to'xtaydi.

Deploydan keyin: nazorat sinovi (*smoke test*) bajariladi: API ishga tushdimi, `/health` javob berdimi, `/predict` oddiy so'rovga javob qaytardimi?

Prezentatsiya rejasi

- 1 Kirish: modeldan ishonchli xizmatga
- 2 Ishlab chiqarish muhitiga joriy etish bosqichlari
- 3 API yaratish va Docker bilan konteynerlashtirish
- 4 Model monitoringi va versiyalash
- 5 CI/CD pipeline
- 6 Xulosa, capstone himoyasi va keyingi yo'l

O'zbek tili uchun NLP: ekotizim va ochiq muammolar

1. Mavjud yo'nalishlar

O'zbek tili uchun ochiq modellar va baholash resurslari paydo bo'lmoqda: UzBERT, BERTbek, UzLiB kabi tashabbuslar.

Lekin ingliz tiliga nisbatan resurslar, benchmarklar va domenga xos tayyor yechimlar hali cheklangan.

Ochiq muammolar

- sifatli yorliqlangan ma'lumotlar kamligi;
- morfologiya va imlo uchun ishonchli vositalar;
- huquq, tibbiyot, ta'lim kabi domenlarga xos modellar;
- standart baholash to'plamlari va benchmarklar.

2. Sizning keyingi hissangiz

Kursda qurgan modullaringiz — amaliy loyiha va ilmiy izlanishlar uchun boshlang'ich poydevor.

Amaliy yo'nalishlar

- 1 kichik, lekin toza o'zbek korpus yoki benchmark yaratish;
- 2 ochiq modelni aniq vazifaga moslab fine-tune qilish;
- 3 RAG yoki agent asosida domenga xos yordamchi qurish;
- 4 model, kod va baholash natijalarini hujjatlashtirib ulashish.

Kalit fikr: kichik, sifatli va qayta ishlatiladigan resurs katta modeldan ham foydaliroq bo'lishi mumkin.

Sintez: notebookdan ishlab chiqarish xizmatigacha

Jihat	Notebook / prototip	Ishlab chiqarish / MLOps
Maqsad	sinash, o'rganish, tajriba qilish	foydalanuvchiga ishonchli xizmat berish
Muhit	shaxsiy kompyuter yoki Kaggle	Docker orqali barqarorlashtirilgan muhit
Ishlash tartibi	qo'lda ishga tushirish	API orqali doimiy xizmat ko'rsatish
Sifat nazorati	bir martalik baholash	monitoring, alert va drift indikatorlari
Yangilash	qo'lda o'zgartirish	CI/CD, model reyestri va rollback

Asosiy g'oya: MLOps notebookdagi modelni **ishonchli**, **takror ishlab bo'ladigan** va **yangilanadigan** xizmatga aylantiradi: *train* → *package* → *serve* → *monitor*, Docker, model reyestri va CI/CD.

Tayanch manba: Sculley va boshqalar (2015)

Manba: Sculley, D., va boshq. (2015). **Hidden Technical Debt in Machine Learning Systems.** *NeurIPS* 28.

Nega bu maqola muhim?

- ML tizimida model kodi faqat kichik qism; ma'lumot, serving, monitoring, konfiguratsiya va test infratuzilmasi ham muhim.
- “Yashirin texnik qarz” — tizim hozir ishlayotgandek ko'rinsa ham, keyin uni saqlash, yangilash va tekshirish qimmatlashib borishi.
- Deploy, monitoring, versiyalash va CI/CD — shu qarzni kamaytirishga yordam beradigan MLOps amaliyotlari.

Muhokama savoli

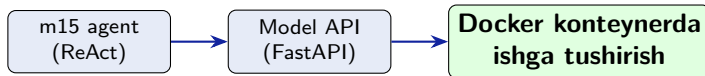
O'zbek tilidagi NLP xizmati uchun qaysi “yashirin texnik qarz” xavfliroq bo'lishi mumkin: **data drift**, **yorliqlangan ma'lumot kamligi** yoki **tokenizer/preprocessing mos kelmasligi**? Nega?

Bugungi maqsadlar bajarildi

- 1 ✓ Modelni ishlab chiqarish muhitiga joriy etish bosqichlarini **tushuntira olasiz**:
train → *package* → *serve* → *monitor*.
- 2 ✓ FastAPI va Docker yordamida modelni API ko'rinishidagi xizmatga aylantirish g'oyasini **tushuntira olasiz**.
- 3 ✓ Monitoring, sifat pasayishi, siljish (*drift*) va model versiyalash nima uchun kerakligini **tushuntira olasiz**.
- 4 ✓ CI/CD pipeline NLP loyihasini avtomatlashtirishda qanday ishlashini va o'zbek NLP ekotizimida keyingi yo'nalishlarni **tushuntira olasiz**.

Yakuniy fikr: notebookdagi model faqat prototip; MLOps uni ishonchli, kuzatiladigan va yangilanadigan xizmatga aylantiradi.

Yakuniy loyiha himoyasi: “O'zbek hujjat yordamchisi”



Himoyada ko'rsating:

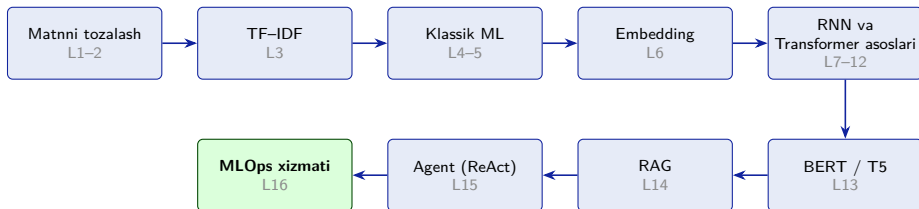
- DocumentAssistantAgent ning jonli ishlashi (o'zbekcha so'rov bilan).
- API orqali modelga murojaat: masalan POST /predict yoki loyiha endpointi.
- Tizimni Docker orqali ishga tushirish va xizmatning barqaror ishlashini qisqa ko'rsatish.

Tayyorlab keling

Bitta ishlaydigan tizimni yaxlit tarzda tushuntiring: **modullar** (m01–m15), **API**, **Docker** va **MLOps** g'oyalari qanday bog'langanini ko'rsating.

Asosiy mezon: alohida qismlar emas, balki **oxirigacha ishlaydigan quvur** (*agent* → *API* → *Docker*) ko'rsatilishi muhim.

Kurs yo'li: matnni tozalashdan ishonchli xizmatgacha



Bitta zanjir — kurs yakuni

Har bir ma'ruza umumiy yo'lning bir qismi edi: **matnni tayyorlash** (L1–6), **modellarni tushunish va o'qitish** (L7–13), **bilimga tayangan tizimlar** (L14–15) va nihoyat **ishonchli xizmat** (L16). Endi sizda to'liq NLP tizimini qurish uchun yaxlit tasavvur bor.

Adabiyotlar va kurs yakuni

- 1 Sculley, D., va boshq. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS 28.
- 2 Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly.
- 3 FastAPI rasmiy hujjatlari va Docker Docs — API yaratish hamda Docker bilan konteynerlashtirish.
- 4 Google Cloud va Microsoft Azure MLOps qo'llanmalari — monitoring, model reyestri va CI/CD amaliyotlari.

Tabriklaymiz!

16 mashg'ulot davomida klassik NLP usullaridan zamonaviy RAG va agent tizimlarigacha bo'lgan yo'lni bosib o'tdingiz. Yakunda o'zbek tilidagi hujjat yordamchisi prototipini API, Docker, monitoring, versiyalash va CI/CD g'oyalari bilan bog'lashni o'rgandingiz.

Keyingi qadam: bu bilimlarni amaliy loyihalarda qo'llash, natijalarni hujjatlashtirish va o'zbek NLP ekotizimiga sifatli resurslar bilan hissa qo'shish.

Ilova S1: model reyestri va A/B sinov

Model reyestri (*model registry*) — model versiyalari va ularning metadatasini saqlash joyi.

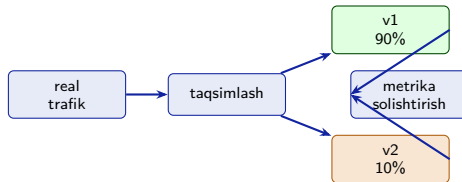
Har bir versiya uchun odatda quyidagilar saqlanadi:

- model artefakti: $v1, v2, \dots$;
- tokenizer/preprocessing va config;
- metrikalar, sana, kod/data versiyasi;
- holat: *staging, production, archived*.

Rollback: yangi versiya yomon ishlasa, xizmat eski barqaror versiyaga tez qaytariladi.

A/B sinov

A/B test — yangi modelni darhol hamma foydalanuvchiga chiqarish emas, balki trafikning kichik qismida sinab ko'rish.



Agar v2 asosiy metrikalarda yaxshi, xavfsizlik metrikalarida esa yomonlashmagan bo'lsa, trafik bosqichma-bosqich oshiriladi; aks holda rollback qilinadi.

Ilova S2: smoke test — deploydan keyingi tezkor tekshiruv

Deploydan keyin xizmat haqiqatan ishga tushganini minimal so'rov bilan tekshiramiz:

```
curl -s -X POST http://localhost:8000/predict \  
  -H "Content-Type: application/json" \  
  -d '{"text": "mahsulot juda yaxshi"}'
```

```
# kutilgan javob:  
# {"label": "ijobiy", "confidence": 0.92}
```

Smoke test — tizimning eng muhim yo'li ishlayotganini tekshiruvchi **minimal nazorat sinovi**.

Tekshiramiz: endpoint javob berdimi, JSON format to'rimi, label va confidence qaytdimi.

CI/CD jarayonida bunday test deploydan keyin avtomatik bajariladi. Agar smoke test o'tmasa, yangi versiya production uchun xavfli deb qaraladi.